

Threads

François Trahay



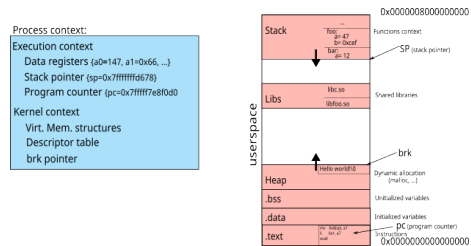
Contents

Threads	1
Contexte d'exécution d'un processus	1
Cloner un processus	2
Flux d'exécution	2
Processus multi-thread	2
Créer un pthread	3
Exemple	3
Terminaison d'un thread	4
Pthread_join	5
Partage de données	5
Accès concurrent aux données	5
Mutex	7

Threads

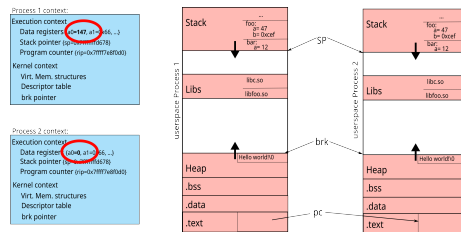
Contexte d'exécution d'un processus

- Contexte: contexte d'exécution + contexte du noyau
- Espace mémoire: code, données, pile



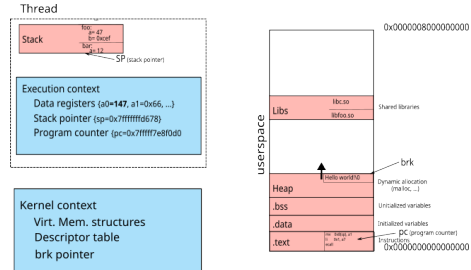
Cloner un processus

- `fork()` crée un nouveau process et duplique le processus appelant
 - Contexte: contexte d'exécution + contexte du noyau
 - Sauf pour le registre `rax` (qui stocke la valeur de retour de `fork`)
 - Espace mémoire: code, données, pile



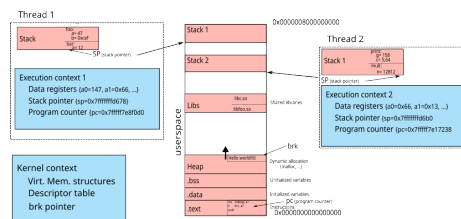
Flux d'exécution

- Une autre manière de représenter un processus est de séparer le flux d'exécution et les ressources du processus
 - Flux d'exécution (ou thread) : contexte d'exécution + pile
 - Ressources: code, données, contexte du noyau



Processus multi-thread

- Plusieurs flux d'exécution
- Des ressources partagées



Dans un processus multi-thread, chaque thread possède un contexte (registres + pile). Le reste de la mémoire (code, données, tas, etc.) et les ressources (fichiers ouverts, etc.) sont partagés entre les threads.

Les piles des threads sont placés en mémoire de manière à pouvoir grandir. Toutefois, si la pile d'un threads grandi trop, elle peut écraser la pile d'un autre thread. Pour éviter ce problème, la taille de la pile est limitée (voir la commande `ulimit -s` qui affiche la taille maximum d'une pile). Cette taille peut être modifiée depuis la ligne de commande (par exemple `ulimit -s 32768`), ou depuis le programme (en utilisant la fonction `setrlimit`).

Créer un pthread

```
int pthread_create(pthread_t *thread,
                  const pthread_attr_t *attr,
                  void *(*start_routine) (void *),
                  void *arg);
```

- Crée un nouveau thread qui exécute la fonction `start_routine(arg)`
- `attr` (in): attributs du thread à créer
- `start_routine` (in): adresse de la fonction à exécuter par le nouveau thread
- `arg` (in): paramètre à passer à la fonction `start_routine`
- `thread` (out): identifiant du thread créé

Nous présentons l'API Pthread (POSIX thread) qui est la plus utilisée en C. Le standard C11 définit une autre interface pour manipuler des threads (voir <https://en.cppreference.com/w/c/thread.html>). Cependant, cette interface n'est pas toujours disponible, et le standard *de facto* reste aujourd'hui l'interface pthread.

Contrairement à la création de processus qui génère une hiérarchie (chaque processus a un processus parent), il n'y a pas de hiérarchie entre les threads.

Exemple

Voici un exemple de programme qui crée 4 threads. La fonction `main` appelle `pthread_create` pour créer un thread qui exécutera la fonction `thread_function` avec le paramètre `parameters[i]`. L'identifiant du nouveau thread est stocké dans `threads[i]`, ce qui permet plus tard à `main` d'attendre la terminaison du thread avec `pthread_join`.

Les threads se terminent à la fin de la fonction `thread_function`.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
```

```

#include <unistd.h>

#define NB_THREADS 4

void* thread_function(void* arg) {

    int *pi = arg;
    int i = *pi;
    printf("Le thread %d démarre\n", i);

    sleep(i+1);

    printf("Le thread %d se termine\n", i);
    return NULL;
}

int main(int argc, char** argv) {

    int parameters[NB_THREADS];
    pthread_t threads[NB_THREADS];

    for(int i=0; i<NB_THREADS; i++) {
        parameters[i] = i;
        pthread_create(&threads[i], NULL, thread_function, &parameters[i]);
    }

    for(int i=0; i<NB_THREADS; i++) {
        void* retval;
        pthread_join(threads[i], &retval);
    }

    return EXIT_SUCCESS;
}

```

Pour compiler le programme, il est nécessaire d'ajouter `-pthread` lors de l'édition de liens.

Terminaison d'un thread

Un thread se termine quand

- il atteint la fin de sa fonction
- il appelle `void pthread_exit(void* retval)`

```
void* thread_function(void* arg) {
```

```

...
if(...) {
    ...
    pthread_exit(NULL); // fin du thread
}
...
return NULL; // fin de thread_function, donc destruction du thread
}

```

Pthread_join

```
int pthread_join(pthread_t tid, void **retval);
```

- Attend la terminaison du thread `tid`.
 - La valeur de retour du thread est stockée dans `*retval`
-

Partage de données

- L'espace mémoire est partagé entre les threads, notamment:
 - les variables globales
 - les variables static local
 - le contexte du noyau (descripteurs de fichiers, flux, signaux, etc.)
 - Quelques ressources ne sont pas partagées
 - les variables locales
 - les registres
-

Accès concurrent aux données

- Plusieurs threads accèdent au même espace mémoire de manière concurrente
- Les structures de données partagées doivent être partagées
 - utilisation de sémaphores ou autres mécanismes (mutex, etc.)

Le programme suivant illustre le problème des accès concurrents à une donnée partagée. Il crée 2 threads qui exécutent chacun `start_routine`. Cette fonction incrémente 100 millions de fois la variable `counter`. Cette variable devrait donc finir par valoir 200 millions.

```

/*
 * compteurBOOM.c
 *
 * Synchronization problem
 */

```

```

*
*/

#include <error.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

#define NBITER 100000000

int counter = 0;

void *start_routine(void *arg) {
    int i;

    for (i = 0; i < NBITER; i++) {
        /* OOPS: WRONG ! Access to an unprotected shared variable */
        counter ++;
    }
    pthread_exit(NULL);
}

int main (int argc, char *argv[]) {
    int rc;
    pthread_t thread1, thread2;

    rc = pthread_create(&thread1, NULL, start_routine, NULL);
    if (rc)
        error(EXIT_FAILURE, rc, "pthread_create");

    rc = pthread_create(&thread2, NULL, start_routine, NULL);
    if (rc)
        error(EXIT_FAILURE, rc, "pthread_create");

    rc = pthread_join(thread1, NULL);
    if (rc)
        error(EXIT_FAILURE, rc, "pthread_join");
    rc = pthread_join(thread2, NULL);
    if (rc)
        error(EXIT_FAILURE, rc, "pthread_join");

    if (counter != 2 * NBITER)
        printf("BOOM! counter = %d\n", counter);
    else
        printf("OK counter = %d\n", counter);
}

```

```
    exit(EXIT_SUCCESS);  
}
```

Pourtant, lorsqu'on exécute ce programme, on observe un comportement différent:

```
$ ./compteurBOOM  
BOOM! counter = 104252638
```

Les modifications concurrentes de `counter` par les deux threads ont “perdu” quelques incrémentations. Pour corriger ce problème, il faut faire en sorte que l'instruction `counter++` soit exécutée en exclusion mutuelle, par exemple en utilisant un sémaphore.

Mutex

Assure l'accès en exclusion mutuelle d'une section critique.

```
pthread_mutex_t lock;  
pthread_mutex_init(&lock, NULL); // Initialise le verrou  
  
pthread_mutex_lock(&lock); // Prend le verrou  
counter++;  
pthread_mutex_unlock(&lock); // Relache le verrou  
  
pthread_mutex_destroy(&lock); // Détruit le verrou
```